File Formats and Tools

- Introduction to next generation sequencing (NGS)
- File types associated with NGS data analysis
  - FASTQ
  - FASTA
  - SAM/BAM
  - GTF/GFF3

CM 515

Learning Objectives

1. Understand next generation sequencing (NGS).
2. Know where to retrieve genomics datasets and information.
3. Identify file types and their contents common to NGS data analysis.
4. Run bioinformatics software for analyzing and visualizing NGS data.
5. Parse files with Python.

NextGen Sequencing

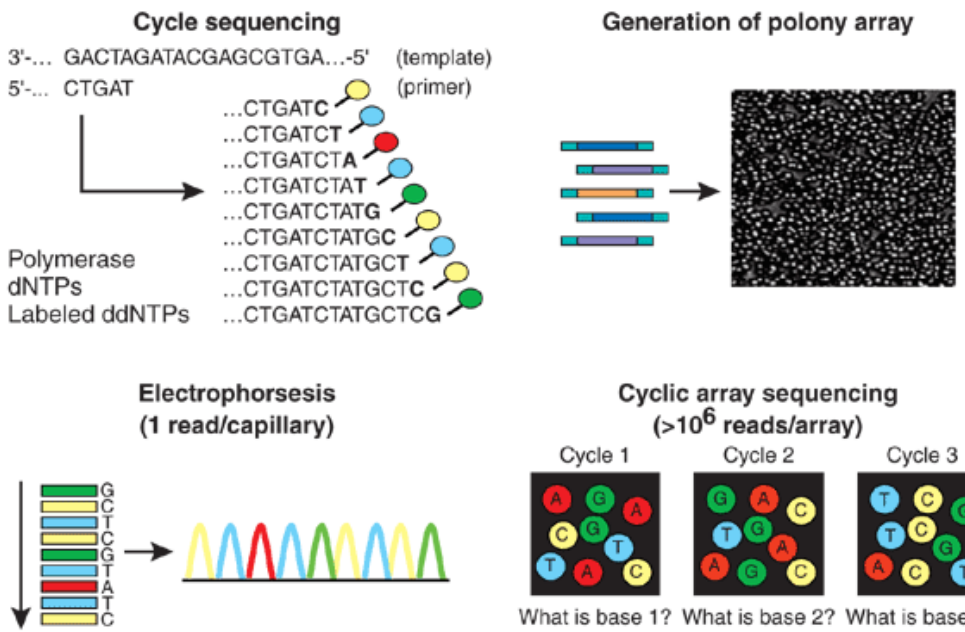
(aka NGS, aka high-throughput sequencing, aka HTS)

Massively parallel sequencing of DNA or RNA

Conventional

Next-Gen

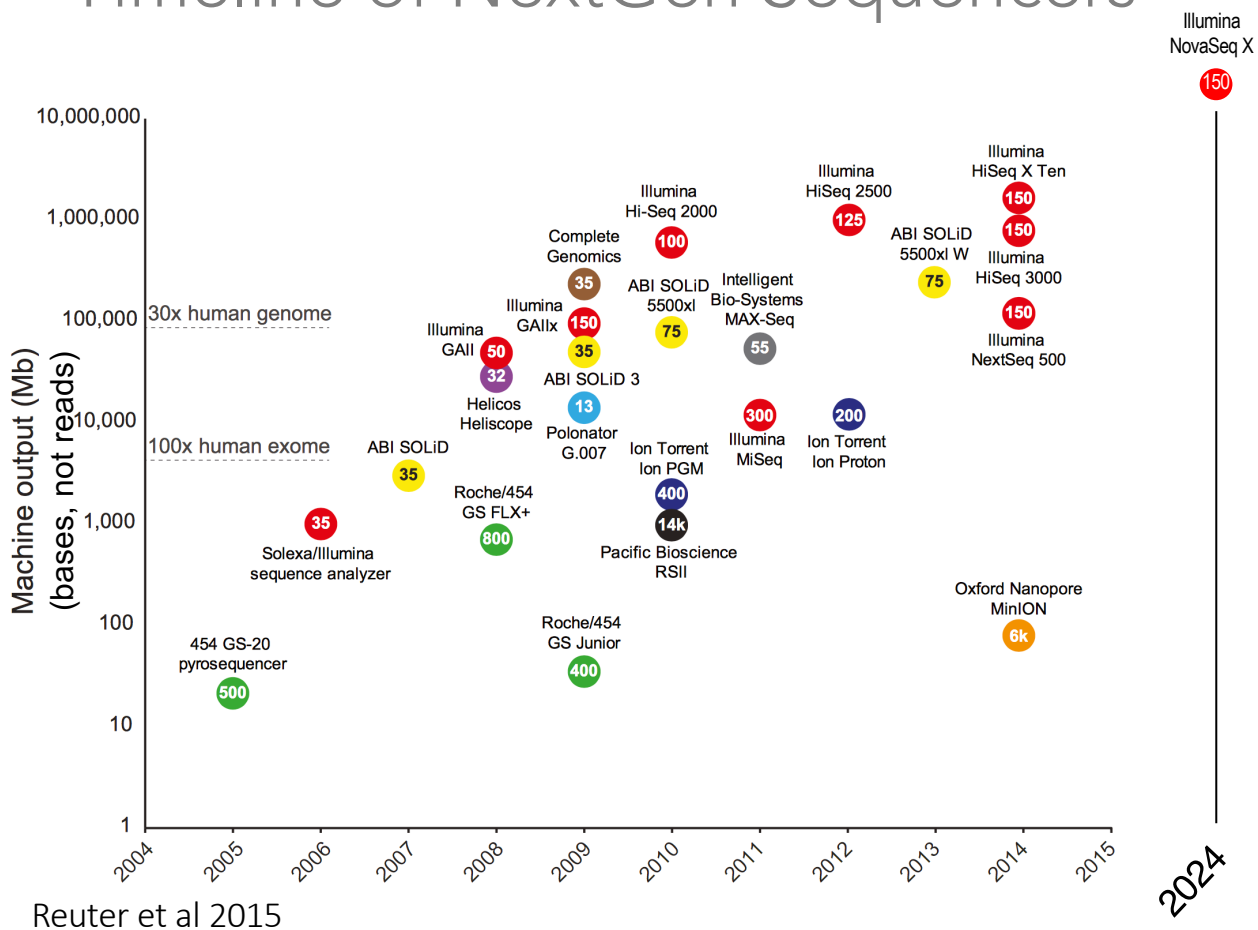
Low throughput
(~1-1,000)



High throughput
(10^6-10^9)

Shendure and Ji (2008)

Timeline of NextGen Sequencers

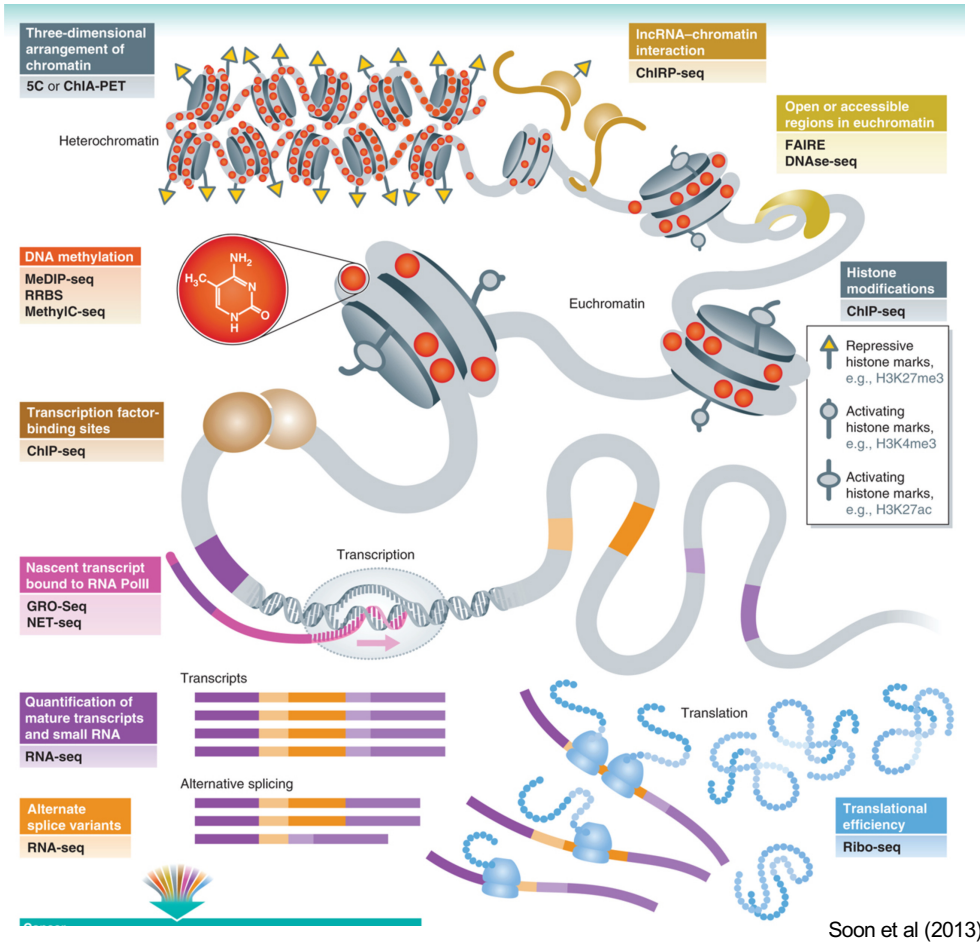


Up to 52 billion reads/run

Illumina dominates the sequencer market

Reuter et al 2015

NextGen Sequencing Applications

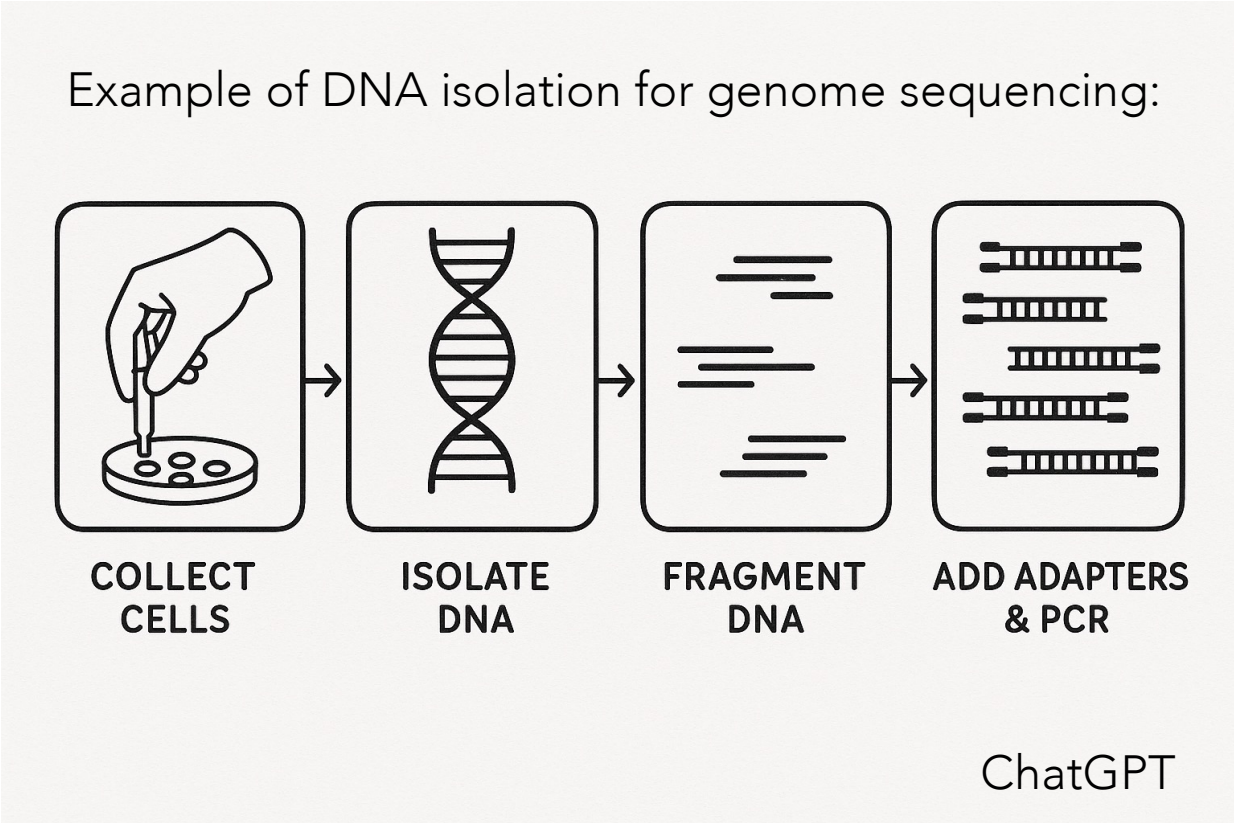


Soon et al (2013)

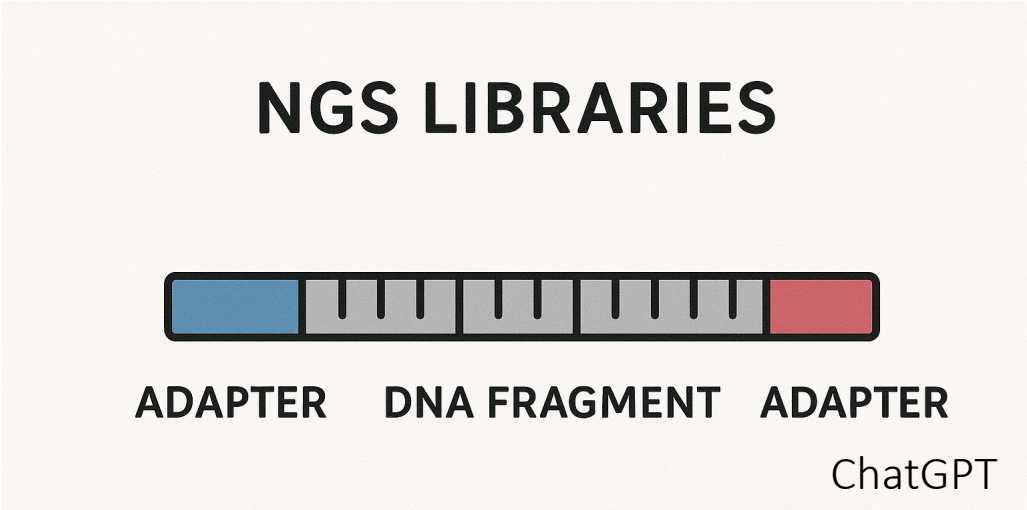
NextGen sequencing can be used to assay many different features of the genome, for example:

- Sequence
- Chromatin marks
- Genome stucture
- Transcription
- Steady state RNA levels
- RNA turnover
- Translation

NextGen Sequencing Library Prep



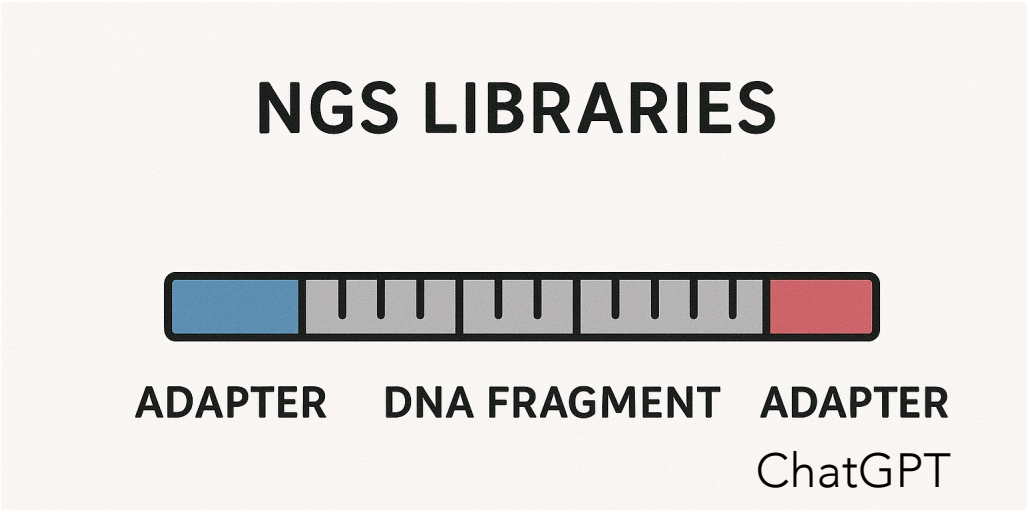
NextGen Sequencing Libraries



Libraries are collections of DNA fragments that contain additional DNA sequences called adapters on both ends that facilitate library preparation and sequencing.

For Illumina sequencing, the DNA fragments themselves are usually 100-500 nt long and can derive from DNA or RNA that has been converted to DNA.

Adapters (aka linkers)



Adapters: sequences added to DNA fragments during library preparation.

1. Contain sequences that allow for mostly unbiased amplification of DNA fragments. During library preparation PCR primers bind to adapters to amplify libraries.
2. Contain sequences that bind to flowcell for sequencing.
3. Contain barcodes to distinguishes libraries produced from different samples.

Illumina Sequencing

Short reads, low error rate, very high-throughput

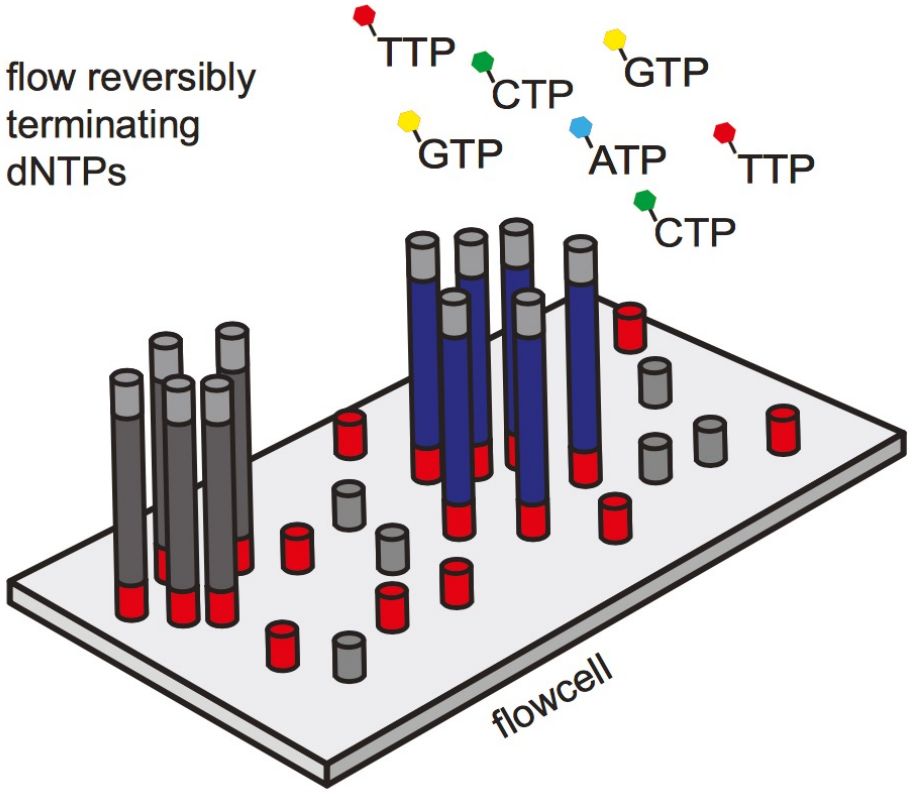


Illustration of
Illumina sequencing.

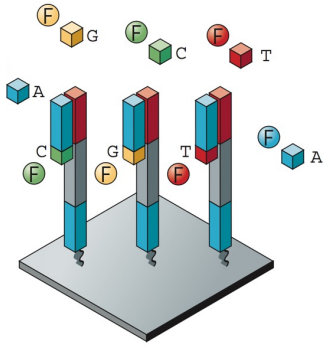
FASTQ Files

FASTQ: The file format typically used for NGS data.

Read 1
```
1 @D64TDFP1:248:C50DMACXX:5:1101:1241:2095 1:N:0:ATCACG
2 CACCGCCCGTCGCTATCCGGGACTGGAATTCTCGGGTGCCAAGGAACTCCA
3 +
4 CCCFFFFFHHHHHJIJGHJJJJIJJJJJGGGFFFFEABDHHHFHFF@@DD>
```

Read 2
```
1 @D64TDFP1:248:C50DMACXX:5:1101:1371:2154 1:N:0:ATCACG
2 TCAATATTTGCATAGGGTATCTGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJGFHIJJJJJJJJJJJJJFHHIIJJHGHJFGHJJI
```

Read 3
```
1 @D64TDFP1:248:C50DMACXX:5:1101:1461:2205 1:N:0:ATCACG
2 GAAAGACGTCTTCCTAGATTATGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJJJJJJJJIJJJJJJJJJHIJJJJJGIIJFGIJJJ
```



Metzker, M.L. (2010) NRG

Line 1: sequence ID, description, and index; begins with @

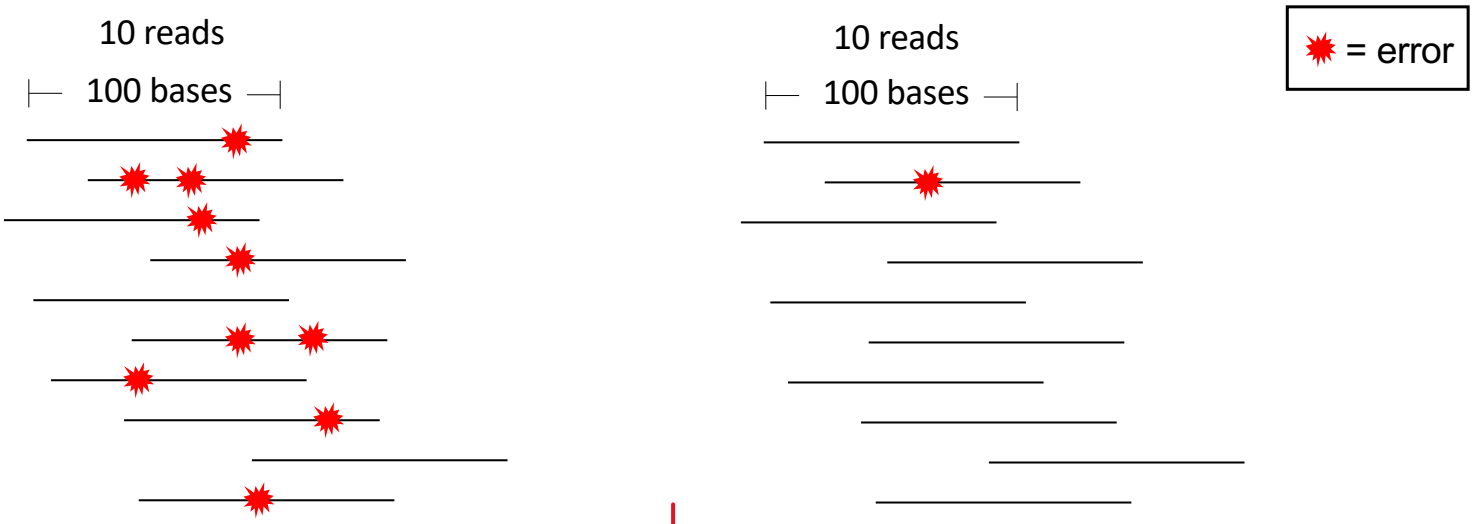
Line 2: sequence; contains only A, C, T, G, and N

Line 3: optional sequence ID; begins with +

Line 4: signal quality of each base, cryptic code

Exercise 1: Visualize FASTQ files

Quality Control: Phred scores



P = 0.01
Q = 20 (Q20)

Q = -10 log(P)

P = ? 0.001
Q = ? 30

- Phred quality score (Q): a measure of the quality of base call during NextGen sequencing.
- P, error probability
- Q30 is a common quality threshold or quality criterion

Quality Control: Assessing Read Quality

Assessing read quality

Phred quality score: a measure of the quality of base calling:
$Q = -10 \log(P)$ where P is the error probability

Phred Quality Score	Probability of incorrect base call	Base call accuracy
10	1 in 10	90%
20	1 in 100	99%
30	1 in 1000	99.9%
40	1 in 10,000	99.99%
50	1 in 100,000	99.999%
60	1 in 1,000,000	99.9999%

Quality Control: Coding within FASTQ files

Read 1

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1241:2095 1:N:0:ATCACG
2 CACCGCCCGTCGCTATCCGGGACTGGAATTCTCGGGTGCCAAGGAACTCCA
3 +
4 CCCFFFFFHHHHHJIJGHJJJJIJJJJJGGGFFFFEABDHHHFHFF@@DD>
```

Read 2

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1371:2154 1:N:0:ATCACG
2 TCAATATTTGCATAGGGTATCTGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJGFHIJJJJJJJJJJJJJFHHIIJJHGHJFGHJJI
```

Read 3

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1461:2205 1:N:0:ATCACG
2 GAAAGACGTCTTCCTAGATTATGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJJJJJJJJIJJJJJJJJJHIJJJJJGIIJFGIJJJ
```

ASCII_BASE=33 Illumina, Ion Torrent, PacBio and Sanger

Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII
0	1.00000	33 !	11	0.07943	44 ,	22	0.00631	55 7	33	0.00050	66 B
1	0.79433	34 "	12	0.06310	45 -	23	0.00501	56 8	34	0.00040	67 C
2	0.63096	35 #	13	0.05012	46 .	24	0.00398	57 9	35	0.00032	68 D
3	0.50119	36 $	14	0.03981	47 /	25	0.00316	58 :	36	0.00025	69 E
4	0.39811	37 %	15	0.03162	48 0	26	0.00251	59 ;	37	0.00020	70 F
5	0.31623	38 &	16	0.02512	49 1	27	0.00200	60 <	38	0.00016	71 G
6	0.25119	39 '	17	0.01995	50 2	28	0.00158	61 =	39	0.00013	72 H
7	0.19953	40 (	18	0.01585	51 3	29	0.00126	62 >	40	0.00010	73 I
8	0.15849	41)	19	0.01259	52 4	30	0.00100	63 ?	41	0.00008	74 J
9	0.12589	42 *	20	0.01000	53 5	31	0.00079	64 @	42	0.00006	75 K
10	0.10000	43 +	21	0.00794	54 6	32	0.00063	65 A			

Quality Control: Removing Low Quality Bases

fastq file snippet

Read x

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1241:2095 1:N:0:ATCACG
2 CACCGCCCGTCGCTATCCGGGACTGGAATTCTCGGGTGCCAAGGAACTCCA
3 +
4 !&&#FFFFHHHHHJIJGHJJJJIJJJJJGGGFFFFEABDHHHFHFF@@!!>
```

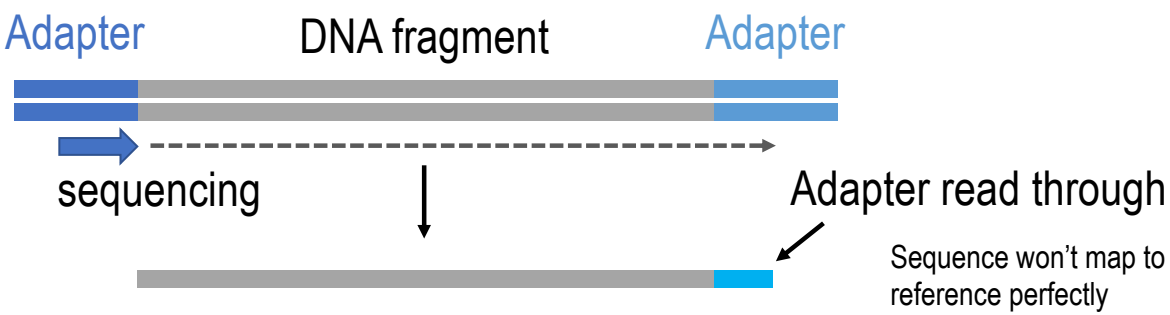
Read y

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1371:2154 1:N:0:ATCACG
2 TCAATATTTGCATAGGGTATCTGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJ!!!!JJJJJJJJJJJJJFHHIIJJHGHJFGHJJI
```

ASCII_BASE=33 Illumina, Ion Torrent, PacBio and Sanger

Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII	Q	P_error	ASCII
0	1.00000	33 !	11	0.07943	44 ,	22	0.00631	55 7	33	0.00050	66 B
1	0.79433	34 "	12	0.06310	45 -	23	0.00501	56 8	34	0.00040	67 C
2	0.63096	35 #	13	0.05012	46 .	24	0.00398	57 9	35	0.00032	68 D
3	0.50119	36 $	14	0.03981	47 /	25	0.00316	58 :	36	0.00025	69 E
4	0.39811	37 %	15	0.03162	48 0	26	0.00251	59 ;	37	0.00020	70 F
5	0.31623	38 &	16	0.02512	49 1	27	0.00200	60 <	38	0.00016	71 G
6	0.25119	39 '	17	0.01995	50 2	28	0.00158	61 =	39	0.00013	72 H
7	0.19953	40 (	18	0.01585	51 3	29	0.00126	62 >	40	0.00010	73 I
8	0.15849	41)	19	0.01259	52 4	30	0.00100	63 ?	41	0.00008	74 J
9	0.12589	42 *	20	0.01000	53 5	31	0.00079	64 @	42	0.00006	75 K
10	0.10000	43 +	21	0.00794	54 6	32	0.00063	65 A			

Quality Control: Removing Adapter Sequence



Read

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1461:2205 1:N:0:ATCACG
2 GAAAGACGTCTTCCTAGATTATGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJJJJJJJJIJJJJJJJJJHIJJJJJGIIJFGIJJJ
```

May need to remove adapter sequence before mapping so the sequence matches the genome.

Read Preprocessing: Trimming and Filtering

Original data (x10 million)

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1241:2095 1:N:0:ATCACG
2 CACCGCCCGTCGCTATCCGGGACTGGAATTCTCGGGTGCCAAGGAACTCCA
3 +
4 !""#FFFFHHHHHJIJGHJJJJIJJJJJGGGFFFFEABDHHHFHFF@@!!>
```

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1371:2154 1:N:0:ATCACG
2 TCAATATTTGCATAGGGTATCTGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJ!!!!JJJJJJJJJJJJJFHHIIJJHGHJFGHJJI
```

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1461:2205 1:N:0:ATCACG
2 GAAAGACGTCTTCCTAGATTATGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 CCCFFFFFHHHHHJJJJJJJJJJJJIJJJJJJJJJHIJJJJJGIIJFGIJJJ
```

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1611:2105 1:N:0:ATCACG
2 TAGCTACGTCTTCACTGGCTATGGAATTCTCGGGTGCCAAGGAACTAACCT
3 +
4 JJJJJJJIHJJHHJJJJJJJJJJJIJJJJJJJJJHIJJJJJGIIJFGIJHH
```

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1611:2105 1:N:0:ATCACG
2 ACACTTTTTCTTTTTTATTATATAAATTTATTTTTATATTATTATTATTTA
3 +
4 !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!GGJHH
```

Trimming and filtering

Processed data (x10 million)

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1241:2095 1:N:0:ATCACG
2 GCCCGTCGCTATCCGGGACTGGAATTCTCGGGTGCCAAGGAACT
3 +
4 FFFFHHHHHJIJGHJJJJIJJJJJGGGFFFFEABDHHHFHFF@@
```

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1371:2154 1:N:0:ATCACG
2 TGGAATTCTCGGGTGCCAAGGAACTCCAGT
3 +
4 JJJJJJJJJJJJJFHHIIJJHGHJFGHJJI
```

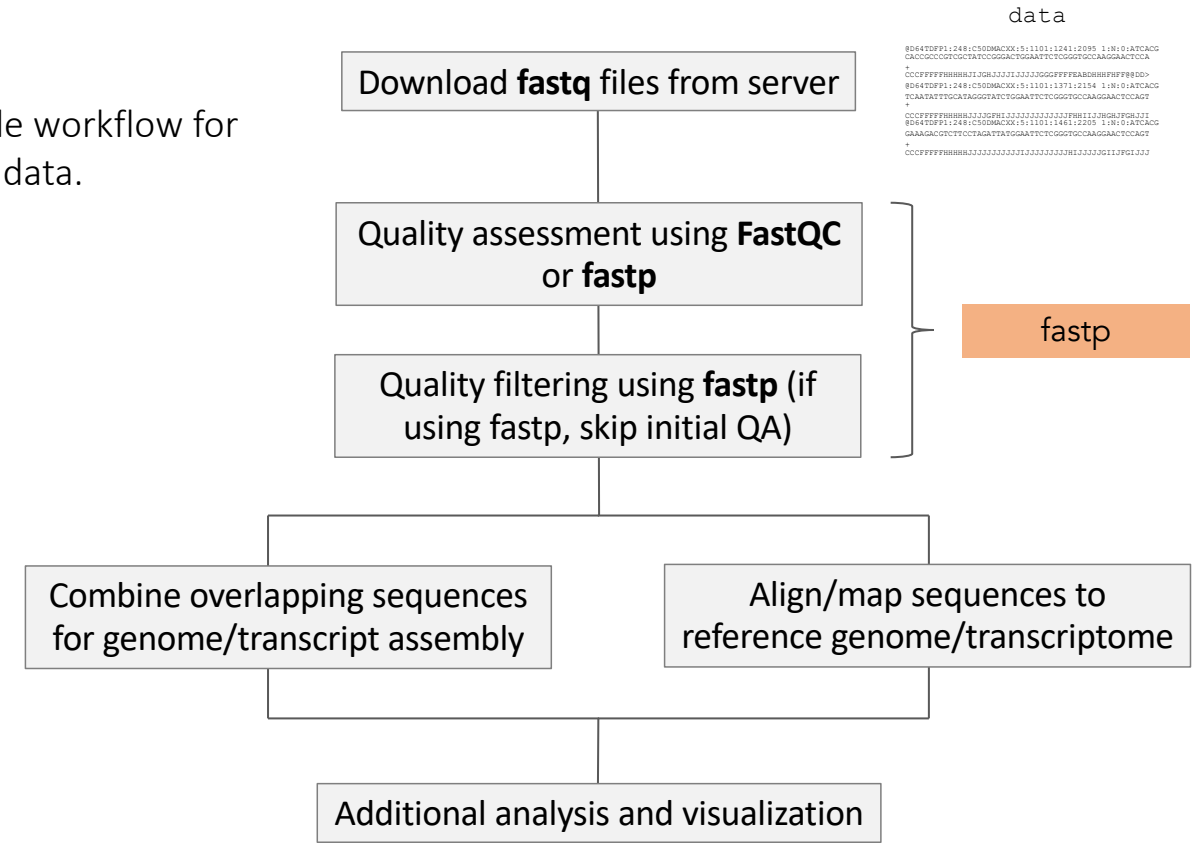
```
1 @D64TDFP1:248:C50DMACXX:5:1101:1461:2205 1:N:0:ATCACG
2 GAAAGACGTCTTCCTAGATTATGGAATTCTCGGGTGCCAA
3 +
4 CCCFFFFFHHHHHJJJJJJJJJJJIJJJJJJJJJHIJJJJ
```

```
1 @D64TDFP1:248:C50DMACXX:5:1101:1611:2105 1:N:0:ATCACG
2 TAGCTACGTCTTCACTGGCTATGGAATTCTCGGGTGCCAAGGAACTAACCT
3 +
4 JJJJJJJIHJJHHJJJJJJJJJJJIJJJJJJJJJHIJJJJJGIIJFGIJHH
```

Discard short sequences

High-throughput Sequencing Data Analysis

A basic example workflow for analyzing NGS data.



Exercise 2: FASTQ file preprocessing with fastp

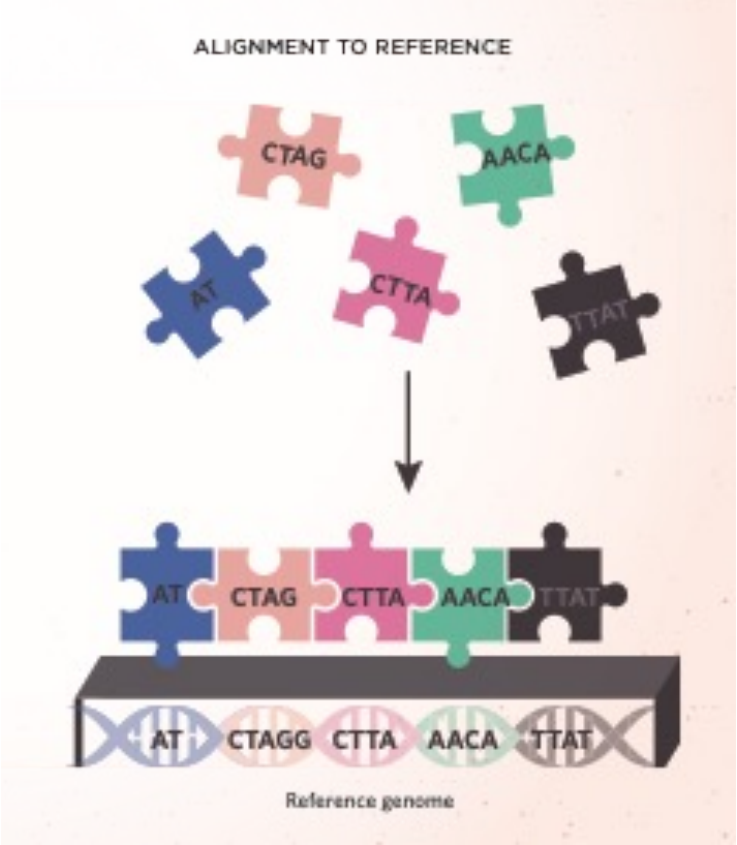
Mapping Reads to a Reference Genome

Mapping reads from NGS data is used to identify where in a sequenced genome or transcriptome each sequence originated.

When sequencing genomic DNA, it is useful assembling genomes and for identifying genetic variation.

When sequencing RNA, it can provide information on sequences and their abundance.

Can you identify a mutation in the reads (puzzle pieces) relative to the reference genome?



Chrisman and Eizenga, The Scientist, 2022

Read Mapping (aka Read Alignment)

Short sequences often align to multiple locations in a genome

- In this example, the read aligns to position on chromosomes 1, 8, and 2.
- Read aligners can report all possible alignments, randomly select 1, or report only alignments that fulfil other criteria, such as quality or number of mismatches.

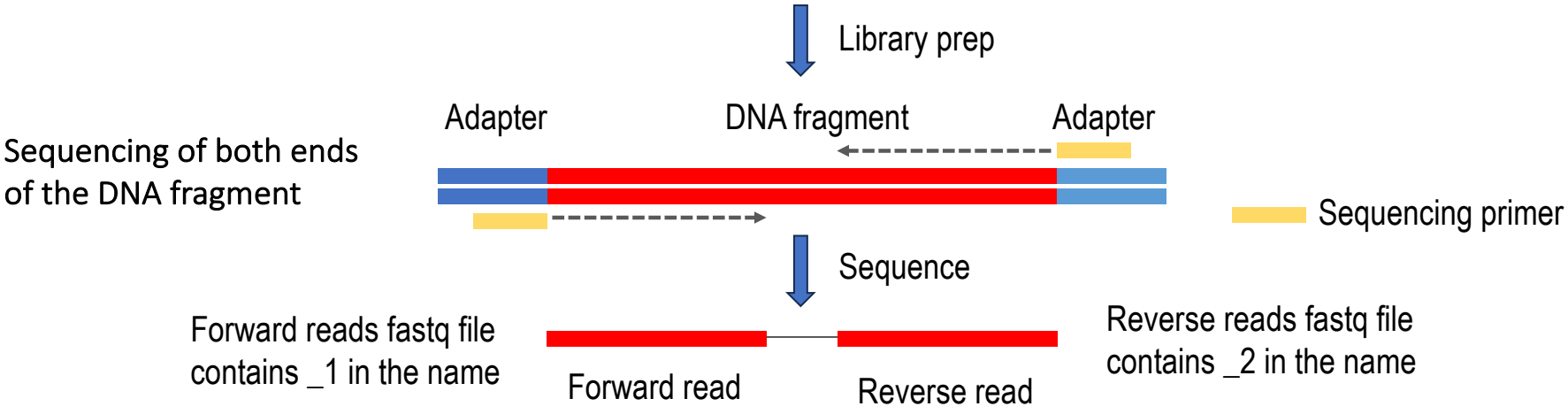
```
                                        Hit 1
                                        TTATCTATCTATCTATCTATCT
                                        |||||||||||||||||||||||
AGGCGACATTTCTATCTATCTTTCATTCTTATCTTATCTATCTATCTATCTATCTTACTCTATTTTCATCGTTAGC
Reference: Chromosome 1
                                        Hit 2
                                        TTATCTATCTATCTATCTATCT
                                        |||||||||||||||||||||||
ACCCGACAAGTCTATGCATCTAACATTCTTACATTATCTATCTATCTATCTATCTAACTCTGCTTTCATGGTTAGC
Reference: Chromosome 8
                                        Hit 3
                                        TTATCTATCTATCTATCTATCT
                                        ||| |||||||||||||||||
AGGCGACATATCTATCAAACTTTCATTCTTATCTTAACTATCTATCTATCTATCTTACTCTATTTTCAACGAAAGC
Reference: Chromosome 2
```

Paired-end and long-read sequencing allow for more confident alignments to repetitive regions of the genome

Paired-end sequencing:

DNA fragment

ACCCGACAAGTCTATGCATCTAACATTCTTACATTATCTATCTATCTATCTATCTAACTCTGCTTTCATGGTTAGC



Paired end sequencing (PE) captures sequence form both ends of the fragment

Paired-end and long-read sequencing allow for more confident alignments to repetitive regions of the genome

In this example, with paired-end sequencing we can now say with confidence which is the correct alignment.

Hit 1

```
ACCCGACAAGTCTATGCATCTA...........TTATCTATCTATCTATCTATCT
|  |||||  |||||  ||||            |||||||||||||||||||||
AGGCGACATTTCTATCTATCTTTCATTCTTATCTTATCTATCTATCTATCTATCTTACTCTATTTTCATCGTTAGC
```

Reference: Chromosome 1

Hit 2

```
ACCCGACAAGTCTATGCATCTA...........TTATCTATCTATCTATCTATCT
|||||||||||||||||||||            |||||||||||||||||||||
ACCCGACAAGTCTATGCATCTAACATTCTTACATTATCTATCTATCTATCTATCTAACTCTGCTTTCATGGTTAGC
```

Reference: Chromosome 8

Hit 3

```
ACCCGACAAGTCTATGCATCTA...........TTATCTATCTATCTATCTATCT
|  |||||  |||||  | ||            ||| |||||||||||||||||
AGGCGACATATCTATCAAACTTTCATTCTTATCTTAACTATCTATCTATCTATCTTACTCTATTTTCAACGAAAGC
```

Reference: Chromosome 2

Read Alignments

Read from _1.fastq read Read from _2.fastq read

```
ACCCGACAAGTCTATGCATCTA...........TTATCTATCTATCTATCTATCT
|||||||||||||||||||||||           |||||||||||||||||||||||
ACCCGACAAGTCTATGCATCTAACATTCTTACATTATCTATCTATCTATCTATCTAACTCTGCTTTCATGGTTAGC
```

Reference: Chromosome 8

Read Name (from fastq): @NS500697:103:HGVJCBGX5:1:11107:13566:4392 1:N:0:GATCAG

Reference alignment name (the name of the sequence): Chromosome 8

Alignment (where the read maps): 1,230,670 nucleotides from start of chromosome 8

Human chromosome 8

What information do read mappers report?

1. Read name
2. Reference alignment name
3. Mapping position (relative to start of the reference sequence)
4. Mapping quality
5. etc

This information is commonly stored in SAM-formatted alignment files.

Genome sequences are often in FASTA format

FASTA: DNA sequence alignment software. The software gave rise to the fasta format, now ubiquitous sequence file format.

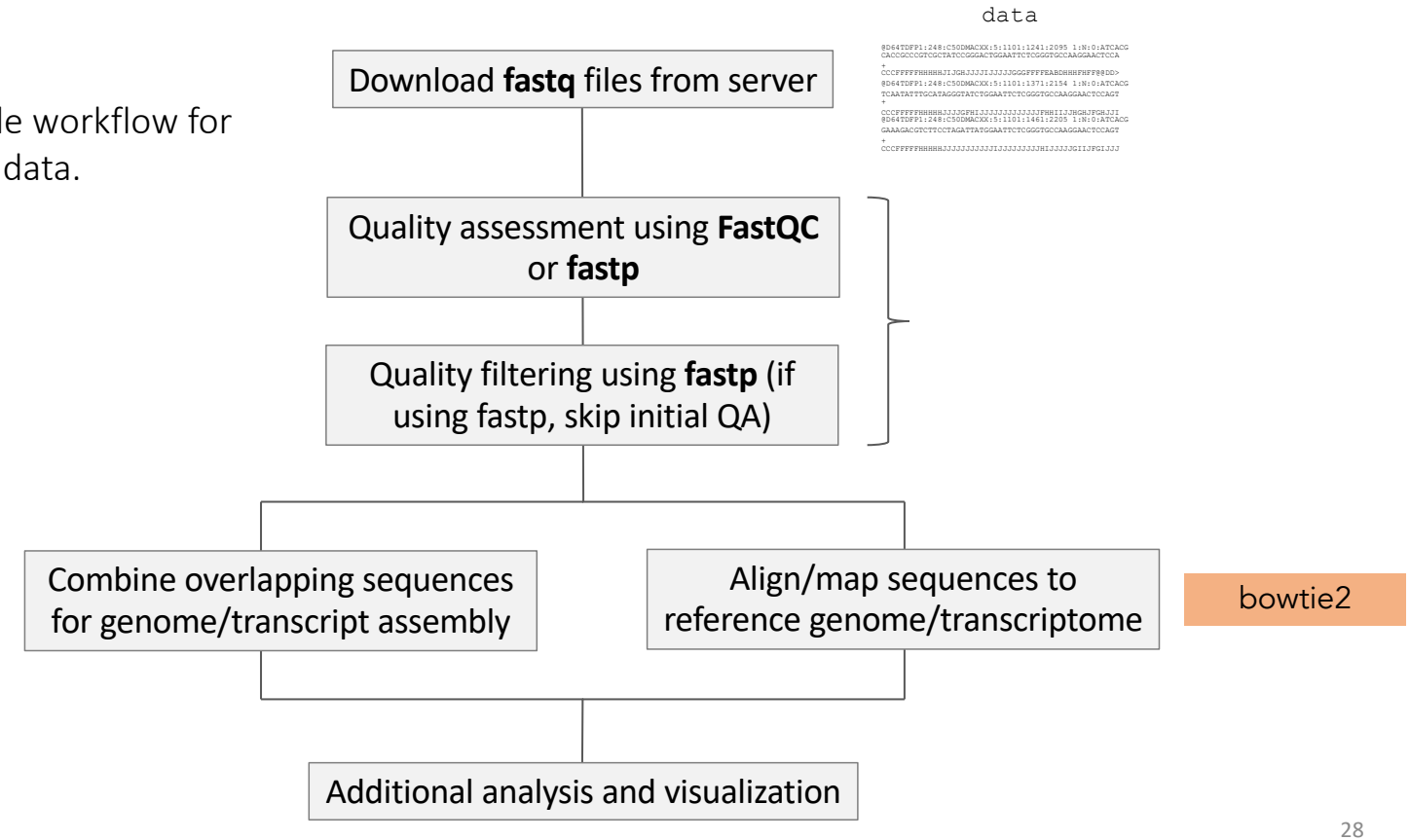
```
>sequence1_description
AGCTAGCATCGACTAGCTACGATCGATCGATCACGAGCTACGACGTAGGCATGGGGGCTTACGATGCTA
CGGCGGAGCTACGGCGACTGCGATCTACGGCGATCGACGGACGGACGTCAGGCGACGATCTATCATCTA
TCGAGCGAGCTACTTACTCTTCTCTATCTACTTATCCCCTTCTTAGGGGTTGATTAGTCTAGCTGGTAC
GATCGAGCGATCTAGAGCGATCGACGAGCTGACGGACGTACTTACTATCGTAGCGACTACTTC

>sequence2_description
CTCTAGCATCGACTAGCTACGATCGATCGATCACGAGCTACGACGTAGGCATGGGGGCTTACGATGCTA
CCCCGGAGCTACGGCGACTGCGATCTACGGCGATCGACGGACGGACGTCAGGCGACGATCTATCATCTA
TCAAGCGAGCTACTTACTCTTCTCTATCTACTTATCCCCTTCTTAGGGGTTGATTAGTCTAGCTGGTAC
GATCTTTCTAGCGAGCGATCTAGAGCGATCGACGAGCTGACGGACGTACTTACTATCGTAGCGACTACT
TC
```

*DNA, RNA, or amino acid sequence can be stored in FASTA format

For more details, see https://en.wikipedia.org/wiki/FASTA_format

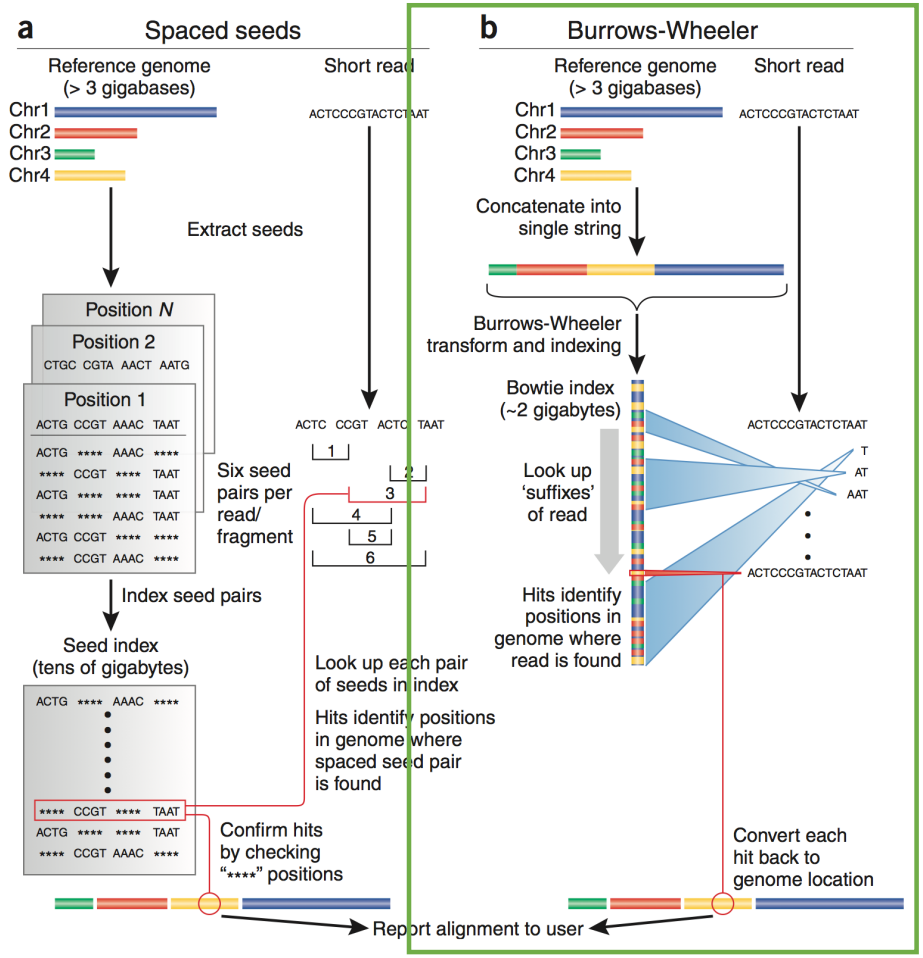
High-throughput Sequencing Data Analysis

A basic example workflow for analyzing NGS data.



Bowtie and Bowtie2

Commonly used programs for read alignment (but not ideal for every application)



Trapnell and Salzberg (2009)

Exercise 3: Read alignment using Bowtie2

SAM/BAM Format

SAMtools: a software package for mining NextGen sequencing data after alignment.

SAM (Sequence Alignment Map): A widely use format for storing alignment data for high-throughput sequencing reads.

BAM (binary SAM): Compressed SAM (binary format).

The file is broken down into two sections:

1. Header section (optional): contains general information about the data such as alignment software used, reference genome aligned againsts, etc. Header lines start with @.

2. Alignment section: contains much of the same information as a fastq file, such as sequence and base quality scores, as well as information about alignment to reference sequence.

For a more complete description, see https://genome.sph.umich.edu/wiki/SAM

SAM/BAM File Format

11 fields + optional 12th TAGs field (not shown in table)

Col	Field	Type	Brief Description
1	QNAME	String	Query template NAME
2	FLAG	Int	bitwise FLAG
3	RNAME	String	References sequence NAME
4	POS	Int	1- based leftmost mapping POSition
5	MAPQ	Int	MAPping Quality
6	CIGAR	String	CIGAR String
7	RNEXT	String	Ref. name of the mate/next read
8	PNEXT	Int	Position of the mate/next read
9	TLEN	Int	observed Template LENgth
10	SEQ	String	segment SEQuence
11	QUAL	String	ASCII of Phred-scaled base QUALity+33

→ A bitwise code with information such as orientation relative to sense strand

→ Information about mismatches and INDELs

Information for PE data

→ The sequence itself

→ The quality scores for each base

Additional fields (12+) contain optional information about the alignment, such as mismatches.

Mapping Quality

In a SAM file, the mapping quality (MAPQ, field 5) is a probability score that indicates the confidence or reliability of each read alignment to a reference sequence. Here's what it signifies:

Scale: MAPQ scores range from 0 to 255, where 255 typically denotes the highest confidence (perfect match) and 0 indicates the lowest confidence (very poor match).

Interpretation:

- 0: Indicates that the read couldn't be confidently mapped to any location in the reference genome.
- **1-60:** Represents a reasonable confidence level, with higher values indicating better alignments.
- **255:** Reserved for cases where the read is mapped uniquely to a single location with no ambiguities.

Usage: SAM/BAM files are often filtered based on MAPQ to exclude poorly mapped reads or to prioritize confidently mapped reads for downstream analysis.

Calculation: MAPQ is derived from the alignment algorithm's assessment of the alignment's quality, considering factors like alignment score, number of mismatches, and the uniqueness of the mapping.

Exercise 4: Parsing sam files with python